

## Function Overloading:

DATE



As we know C++ is object-oriented programming language. Means it provides us features to implement the principle of oops.

And **Polymorphism** is one of the principle of oops.

### Polymorphism:

To take more than one form

or

polymorphism allows us to write more flexible and reusable code by enabling objects of different types to be treated as objects of common base.

There are two main types of polymorphism in C++.

- Compile-time polymorphism (static polymorphism) & Overloading?

**Function overloading**

**Operator overloading**

→ Achieved through Overloading

- Run-time polymorphism (dynamic polymorphism) & Overriding?

**Virtual function**

**Inheritance function**

→ member function of a base class that is available in the derived class through inheritance.

And Run-time polymorphism is achieved through Overriding.





## function Overloading :

- Function overloading is a way of implementing **polymorphism** in C++
- Overloading refers to the use of the same thing for different purposes.

\* Use of same function name to create functions that perform a variety of different tasks is known as **function Overloading** in OOP.

Example :

int add(int a, int b); // prototype 1

int add(int a, int b, int c); // prototype 2

double add(double x, double y); // prototype 3

double add(int p, double q); // prototype 4

double add(double p, int q); // prototype 5

cout << add(5, 10); // uses prototype 1

cout << add(15, 10.0); // uses prototype 4

cout << add(12.5, 7.5); // uses prototype 3

cout << add(5, 10, 15); // uses prototype 2

cout << add(0.75, 5); // uses prototype 5

## How function Overloading is Resolved?

A function call first matches the **prototype** having the same number and type of **arguments** and then calls the appropriate **function** for execution.

The function selection involves the following steps :





**Step-1** The compiler first tries to find an exact match in which the types of **actual arguments** are the same and use that function.

**Step-2** If an exact match is not found, the **compiler** uses the integral promotions to the **actual arguments** such as

**char** to **int**  
**float** to **double**